

CUCEI



UdeG

Algoritmo Genético Continuo

13 de Septiembre de 2026

Algoritmos Metaheurísticos

Maestra: Paredes López Ángel Ignacio

Alumno: Mariscal Rodríguez Omar Jesús (220858478)

Índice

Indicaciones Generales:	3
Diseño del Algoritmo:	5
Sintonización de Parámetros:	8
Análisis Comparativo (Ackley):	10
Discusión Analítica:	15
Anexo del Código:	16
Conclusión:	61

INDICACIONES GENERALES:

Tras comprender las limitaciones de la discretización en el Algoritmo Genético Binario (AGB), en esta práctica se implementará un Algoritmo Genético Continuo, programado en MATLAB, C++ o Python. Al utilizar variables de punto flotante, los operadores de cruce y mutación tradicionales ya no son válidos, por lo que deberá investigar e implementar operadores continuos (ej. cruce uniforme y mutación gaussiana). El objetivo principal es analizar la capacidad del algoritmo para evadir mínimos locales y escalar a problemas de mayor dimensionalidad.

Funciones a Optimizar:

1. **Función de Ackley (2D):** *(Para comparación directa con la Actividad 2)*

$$f(x, y) = e - 20 \exp\left(-0.2 \sqrt{\frac{x^2 + y^2}{2}}\right) - \exp\left(\frac{\cos(2\pi x) + \cos(2\pi y)}{2}\right) + 20, \quad x, y \in [-5, 5]$$

2. **Función Esfera (N-dimensional):**

$$f(x) = \sum_{i=0}^n x_i^2, \quad x \in [-5.12, 5.12]$$

Metodología y Configuración de Parámetros: A diferencia de la actividad anterior, **no se proporcionan valores fijos** para el tamaño de población, probabilidad de cruce (P_c) o probabilidad de mutación (P_m). Es responsabilidad del estudiante realizar un proceso de sintonización empírica (*parameter tuning*) para encontrar una configuración que ofrezca un buen balance entre exploración (evitar convergencia prematura en Ackley) y explotación (precisión en la Esfera).

Para garantizar validez estadística, todo reporte de resultados debe basarse en la media de al menos **30 ejecuciones independientes** por experimento.

Entregables del Reporte (PDF):

- **Diseño del Algoritmo:** Explique matemáticamente cómo funcionan los operadores de cruce y mutación continua que decidió implementar en su código (No inserte el código aquí, explique la lógica matemática).
- **Sintonización de Parámetros:** Una tabla o gráfica que muestre brevemente qué valores de P_c , P_m y tamaño de población probó y cuáles eligió como los mejores. Justifique su elección.
- **Análisis Comparativo (Ackley):** Compare los resultados estadísticos (Mejor, Media, Desviación Estándar), obtenido en esta actividad con los de su Actividad 2 (AGB de 8 bits) ¿Qué diferencias observa en la precisión final y en la velocidad de convergencia?
- **Prueba de Escalabilidad (Esfera):** Si intentáramos resolver la función Esfera de 10 dimensiones utilizando el Algoritmo Binario de la Actividad 2 con una

resolución de 16 bits por variable ¿De qué tamaño sería el espacio de búsqueda total (cuántas combinaciones binarias posibles existirían)? Con base a este cálculo, explique por qué la codificación continua es preferible para problemas multivariantes.

- **Anexo del Código:** Adjuntar en el reporte el código fuente debidamente comentado
- **Conclusión:** Describir los resultados y conclusiones finales de la actividad.
- **Evaluación Presencial y Modificación del Código:** El 40% de los puntos de esta actividad se acreditarán mediante la modificación presencial del código en clase, la cual se realizará exclusivamente el día 21 de septiembre (una clase antes del límite de entrega del reporte)

DISEÑO DEL ALGORITMO

Arquitectura del Código:

Explicando brevemente la arquitectura del código, está diseñado siguiendo los mismos pasos de la actividad anterior, con el Patrón de Estrategia para facilitar los cambios en ciertos procesos específicos del algoritmo (como los cruces, mutaciones y/o selecciones), ya con un conjunto de estos que pueden ser acoplables mediante líneas de comando o agregar otros siguiendo los Protocolos o Contratos. Desde este punto de vista, el código se divide en dos archivos como el AGB: El `ga_continuo.py` que codifica las configuraciones, almacena los problemas, tiene las estructuras en las que recopilamos la información desde las especificaciones del problema hasta los resultados del algoritmo. Tiene a su vez el motor que corre el algoritmo con todo lo que involucra las configuraciones. Por su parte, el `main.py` hace toda la tarea de orquestador al recibir las configuraciones generales elegidas desde línea de comandos, crear el motor de configuraciones y ejecutar el algoritmo las veces que sean necesarias, calculando las estadísticas y graficando.

Hasta ahora, muy parecido al código de la tarea anterior, pero la distinción más importante es que el `main.py` incluye dos modos de ejecución, el modo **final** que es la ejecución tradicional y el modo **tuning**. Este último modo está programado para encontrar una combinación de hiper parámetros ideal en base a múltiples ejecuciones más ligeras (configurables pero establecidas en 10 vueltas del algoritmo) con múltiples configuraciones diferentes que podemos introducir de una sola vez vía consola (n número de población, Pc y Pm por ejemplo), el algoritmo se ejecuta con cada una de las combinaciones y configuraciones en dos problemas distintos y nos regresa una tabla similar a la que nos daría la ejecución convencional, en el que podemos comparar cuál de estas configuraciones parece ir mejor en esta exploración preliminar para posteriormente, pasar al modo **final** para probar en el escenario real y completo dicha configuración. Más adelante, en la elección de los hiper parámetros rescataré esta modalidad.

Funcionamiento de los Operadores:

El código actualmente, por su patrón de estrategia permite tener varias implementaciones de operadores diferentes y ser configurable desde consola o agregar varias sin modificar tanto el código, tanto para selección, mutación, cruce, criterios de paro y el manejo de límites. La lista de estrategias u operadores que actualmente el código implementa son las siguientes:

Operadores de Selección:

- Ruleta
- Torneo

Operadores de Cruce:

- Aritmético

- Blx Alpha
- **Uniforme**

Operadores de Mutación:

- **Gaussiano**
- Reseteo Uniforme

Manejo de Límites:

- **Acote**
- Reflejo
- Remuestreo

Aquellos en negritas en la lista son los operadores por defecto del código, así que son en los que me centraré en esta subsección, sin contar la selección por torneo que ya venimos tratando desde la actividad anterior.

Nota: Cabe mencionar que el código que se anexa en su propia sección está comentado en cada parte, incluyendo la idea matemática detrás de cada una de estas operaciones.

Operador de Cruce Uniforme

En el cruce uniforme, cada gen de los descendientes se decide de manera independiente con una probabilidad fija (usualmente $p = 0.5$). Dados dos padres $\mathbf{P}_1$ y $\mathbf{P}_2$, el vector del primer hijo $\mathbf{H}_1$ en la posición i se define matemáticamente mediante un vector máscara aleatorio $u_i \sim U(0, 1)$:

$$\mathbf{H}_1^{(i)} = \mathbf{P}_1^{(i)} \text{ si } u_i < 0.5, \text{ de lo contrario } \mathbf{P}_2^{(i)}$$

$$\mathbf{H}_2^{(i)} = \mathbf{P}_2^{(i)} \text{ si } u_i < 0.5, \text{ de lo contrario } \mathbf{P}_1^{(i)}$$

Ejemplo: Sean dos padres bidimensionales $\mathbf{P}_1 = [2.5, -1.2]$ y $\mathbf{P}_2 = [-0.8, 4.1]$. Si se genera una máscara aleatoria $\mathbf{u} = [0.2, 0.7]$:

- Para $i = 1$: Como $u_1 = 0.2 < 0.5$, $\mathbf{H}_1^{(1)}$ hereda de $\mathbf{P}_1^{(1)} = 2.5$ y $\mathbf{H}_2^{(1)}$ de $\mathbf{P}_2^{(1)} = -0.8$.
- Para $i = 2$: Como $u_2 = 0.7 \geq 0.5$, $\mathbf{H}_1^{(2)}$ hereda de $\mathbf{P}_2^{(2)} = 4.1$ y $\mathbf{H}_2^{(2)}$ de $\mathbf{P}_1^{(2)} = -1.2$.

Resultado final: $\mathbf{H}_1 = [2.5, 4.1]$ y $\mathbf{H}_2 = [-0.8, -1.2]$.

Operador de Mutación Gaussiana

En este operador, cada gen x_i del individuo se muta con una probabilidad $P_{\square}$. La mutación añade un ruido extraído de una distribución normal con media cero $N(0, \sigma^2)$, donde la desviación estándar σ se escala según el rango de la variable ($\max_i - \min_i$) y un parámetro global α :

$$x_i' = x_i + \Delta_i, \text{ donde } \Delta_i \sim N(0, (\alpha \cdot (\max_i - \min_i))^2)$$

Ejemplo: Suponga una variable con rango $[-32.768, 32.768]$ (ancho del dominio = 65.536), un valor actual $x_1 = 5.0$, y un factor de escala $\alpha = 0.1$. La desviación estándar resulta en $\sigma = 0.1 \times 65.536 = 6.5536$.

- Si la probabilidad determina que el gen muta y se obtiene una muestra aleatoria gaussiana $\Delta_1 = -1.34$:

$$x_1' = 5.0 + (-1.34) = 3.66$$

De esta forma, la mutación realiza pequeñas perturbaciones locales que facilitan el ajuste fino alrededor de soluciones prometedoras.

Ahora, los operadores de cruce y mutación en el dominio real nos traen cierto problema de que nuestro gen pueda salirse del dominio tipo caja que los problemas exigen, se salen del espacio de búsqueda y aunque encontremos mejores soluciones allá. no incuben a nuestro problema. De tal manera, también tenemos que tener una estrategia cuando pase eso, y es lo que hace el manejo de límites:

Manejo de Límites por Acote (Clamping):

Dado que los operadores continuos pueden generar valores fuera del espacio de búsqueda permitido $[\min_i, \max_i]$, el método de acote proyecta cualquier valor excedente directamente hacia el frontera más cercana mediante la función de saturación:

$$x_i'' = \min(\max(x_i', \min_i), \max_i)$$

Ejemplo: Para una variable acotada en el intervalo $[-32.768, 32.768]$:

- Caso 1: Si tras la mutación $x_1' = 35.12$ (supera el límite superior), se ajusta a $x_1'' = 32.768$.
- Caso 2: Si $x_2' = -40.50$ (supera el límite inferior), se ajusta a $x_2'' = -32.768$.

Caso 3: Si $x_3' = 12.40$ (dentro del rango), permanece sin cambios: $x_3'' = 12.40$.

SINTONIZACIÓN DE PARÁMETROS

Para la elección de hiper parámetros, se utilizó el modo tuning antes mencionado. Justamente para garantizar un equilibrio óptimo entre la exploración (necesaria para intentar evadir los mínimos locales de la función Ackley) y la explotación (necesaria para alcanzar la precisión de la función Esfera), se diseñó un experimento modo tuning de sintonización mediante una búsqueda de cuadrícula. Se evaluaron 27 combinaciones distintas utilizando 10 corridas independientes por cada configuración, variando el tamaño de la población $\in \{30, 60, 100\}$, la Probabilidad de Cruza $P_c \in \{0.70, 0.85, 0.95\}$ y la Probabilidad de Mutación $P_m \in \{0.02, 0.10, 0.20\}$.

A continuación, se presenta un extracto representativo con las mejores combinaciones obtenidas y las de menor rendimiento:

Población	Pc	Pm	Éxito (Ackley)	Éxito (Esfera)	Mejor Fitness (Ackley)	Mejor Fitness (Esfera)
60	0.95	0.10	100%	100%	0.00e+00	0.00e+00
100	0.85	0.10	100%	100%	0.00e+00	0.00e+00
100	0.95	0.02	100%	100%	0.00e+00	0.00e+00
...	...	...	...	...	...	...
30	0.70	0.02	80%	90%	5.00e-03	3.58e-04
30	0.85	0.02	70%	100%	5.40e-03	1.13e-04

Justo como en la actividad pasada, el porcentaje de éxito se calcula viendo al final de los algoritmos en qué proporción se logró llegar al mínimo óptimo analítico.

Tras analizar los resultados, se descartó el tamaño de población de 30 individuos debido a que mostró una grave propensión a la convergencia prematura (tasas de éxito del 70% al 80% en Ackley), evidenciando una falta de diversidad genética inicial. Por otro lado, aunque la población de 100 individuos garantizó una tasa de éxito del 100% en casi todos los escenarios, representa un costo computacional innecesario.

Respecto a la mutación, se observó que un Pm agresivo (0.20) inyectaba demasiado ruido al sistema; si bien no afectó drásticamente a Ackley, retrasó severamente la convergencia en la función Esfera, impidiendo la exploración fina de la solución.

Configuración Final Elegida:

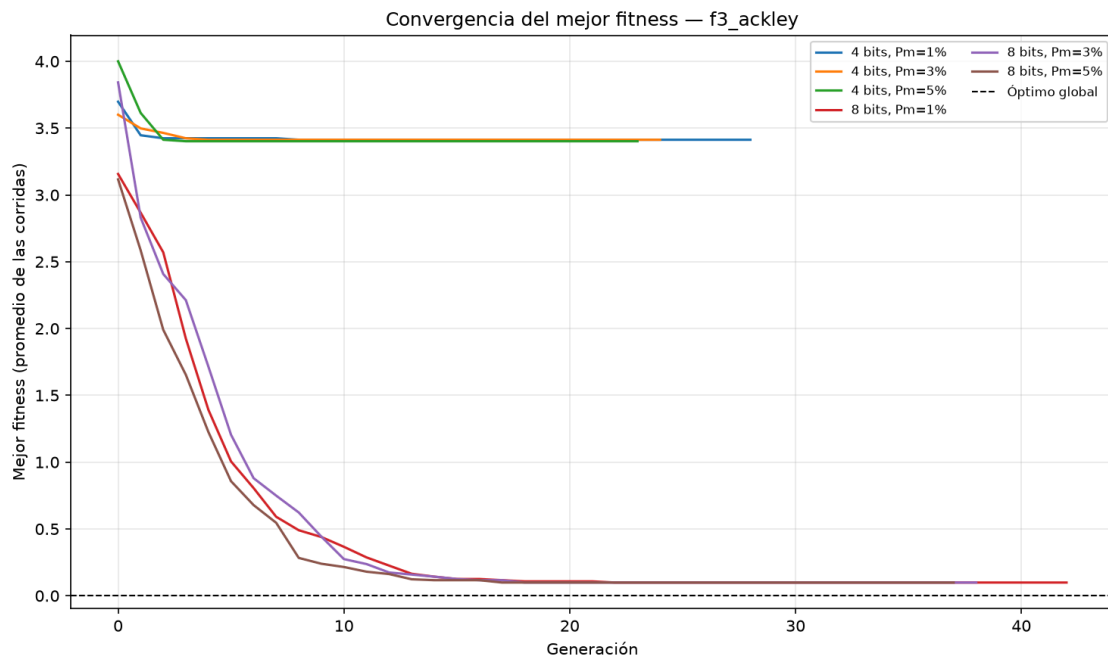
Se determinó una configuración ideal como: Tamaño de población = 60, una Probabilidad de Cruza (P_c) = 0.95 y una Probabilidad de Mutación Gaussiana (P_m) = 0.10

Esta configuración logró una efectividad del 100% tocando el 0.00 exacto en ambos ecosistemas. El alto P_c permitió una rápida convergencia, mientras que el P_m del 10% actuó para escapar de los mínimos locales de Ackley sin destruir el refinamiento de la función esfera. Además, al requerir menos de 80 generaciones promedio para converger, probó ser eficiente.

ANÁLISIS COMPARATIVO (ACKLEY)

Para evaluar objetivamente el impacto de la representación genética, se contrataron los resultados estadísticos obtenidos en la optimización de la función de Ackley utilizando el Algoritmo Genético Continuo frente al Algoritmo Genético Binario de 8 bits de la actividad 2.

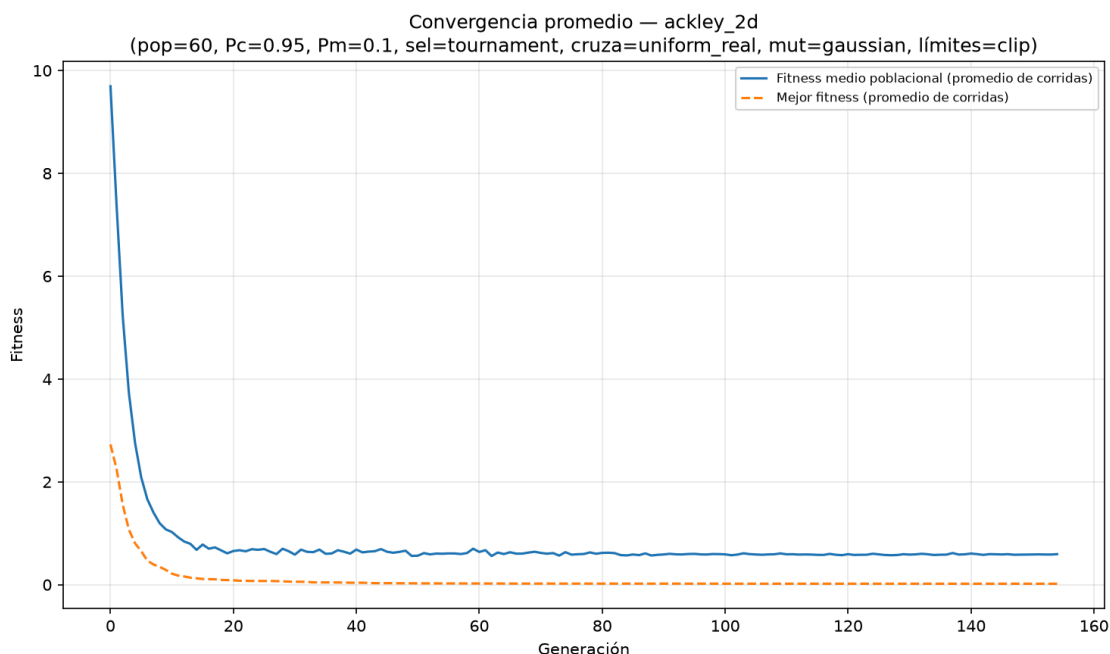
Resultados de la función Ackley en AGB:



Mejores resultado (Pm 0.01)

Bits	Mejor	Peor	Media	Desviación Estándar	Tasa de Éxito
8	0.0988030933	0.0988030933	0.0988030933	0	0

Resultados de la función Ackley en AGC:



Mejor	Peor	Media	Desviación Estándar	Tasa de Éxito
0.000576 2584	0.1211851 991	0.01956041 68	0.0249612640	0.0333

Precisión Final

La diferencia más drástica entre ambos algoritmos radica en la precisión que el AGC es capaz de lograr. El mejor fitness del AGB de 8 bits alcanzó 0.098803. El algoritmo quedó físicamente incapacitado para mejorar este valor debido a que la cuadrícula de discretización no poseía un escalón más cercano al óptimo real. En contraste, el AGC logró un mejor fitness de 0.000576 lo que representa una precisión de alrededor de 171 veces mejor.

Velocidad y Comportamiento de Convergencia

Al analizar las curvas de convergencia, el cambio de paradigma es evidente, la convergencia del AGB desciende rápidamente pero colisiona contra la ‘pared de resolución’ alrededor de la generación 20, transformando la curva en una línea horizontal plana y prematura. Con el AGC, el descenso del error es casi vertical durante las 10 primeras generaciones. En lugar de chocar contra un muro, el AGC entra en una

fase de refinamiento asintótico donde continúa puliendo los decimales generación tras generación hasta que el criterio de paro por estancamiento detiene el proceso en la generación promedio 76.4

Resulta imperativo justificar que el 3% de tasa de éxito obtenido en este experimento específico frente a las altas tasas logradas durante la sincronización. Esta caída es un reflejo directo de la naturaleza del operador de cruce elegido para la corrida fue el uniforme real. A diferencia de la cruce BLX - alpha (que crea hijos en un espacio extendido entre y más allá de los padres), la cruce uniforme real simplemente intercambia coordenadas exactas.

PRUEBA DE ESCALABILIDAD (ESFERA)

Para evaluar la robustez del AGC frente al incremento de variables, se optimizó la función Esfera en dos escenarios distintos: 4 dimensiones y 10-dimensiones tal como se pedía. Ambos experimentos utilizaron la misma configuración antes elegida. A continuación, los resultados:

Problema	Mejor	Peor	Media	Desviación Estándar	Tasa de Éxito	Generación media
Esfera 4D	0.0000021466	0.0007186314	0.0001012883	0.0001605527	1	121.8
Esfera 10D	0.0001165886	0.0037084022	0.0011228754	0.0008234147	0.5333	225

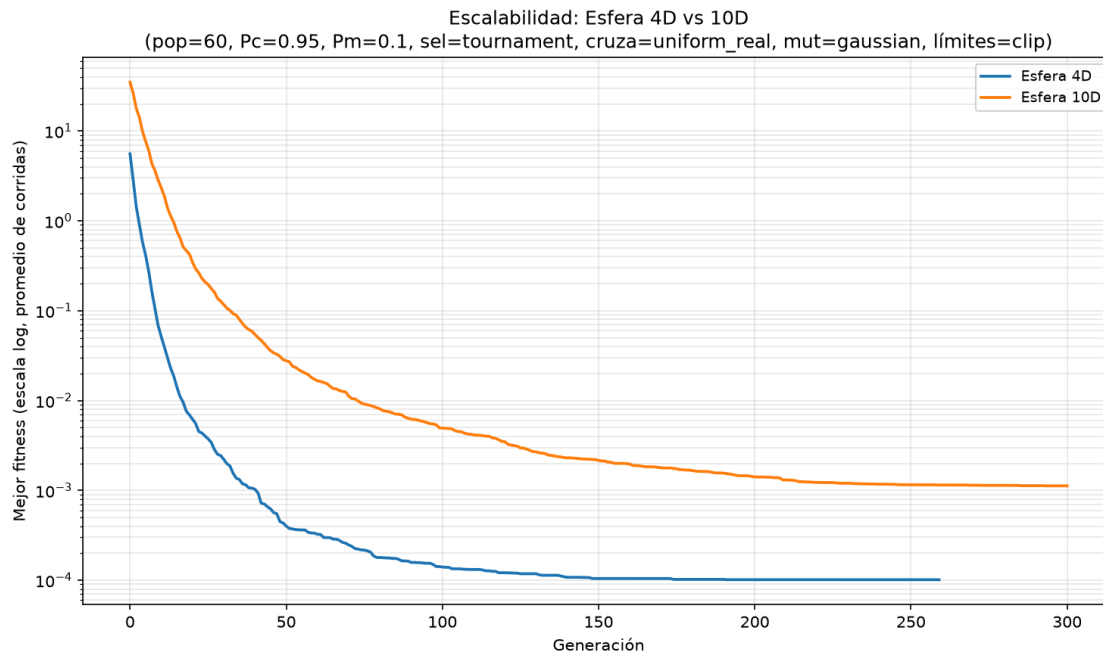
Degradación del Rendimiento

Al contrastar los resultados de las 30 corridas independientes, el impacto de escalar 6 dimensiones es severo. Mientras que en 4D el algoritmo resolvió la función con 100% de éxito a una tolerancia de 10^{-3} , al pasar a 10D, el éxito se desplomó a un 53%.

El mejor individuo histórico en 4D alcanzó un error casi nulo de 2×10^{-6} . En 10D, la precisión se degradó en dos órdenes de magnitud, logrando un mejor fitness de 1.17×10^{-4} .

El algoritmo requirió casi el doble de generaciones promedio para activar el criterio de paro por estancamiento en el entorno de 10 dimensiones (255 generaciones frente a las 121.8 del entorno 4D)

Curvas de Convergencia



La gráfica ilustra visualmente el fenómeno de la Maldición de la dimensionalidad. Mientras que la 4D muestra un descenso agresivo y continuo, penetrando la barrera de 10^{-3} antes de la generación 50 y estabilizándose a un nivel de alta precisión, la curva de 10D comienza con un error inicial mucho mayor. Su descenso es bastante más lento y lo más crítico es que su asíntota de convergencia se aplana en forma prematura cerca de la generación 200, deteniéndose en la franja de 10^{-3} .

Al pasar de 4 a 10 dimensiones, el volumen del dominio espacial crece de forma hiper-exponencial. Una población fija de 60 individuos posee suficiente densidad genética para barrer eficientemente un hipercubo de 4 dimensiones; sin embargo, en 10 dimensiones, esos mismos 60 individuos quedan ridículamente dispersos. La distancia entre las soluciones candidatas aumenta masivamente, obligando al algoritmo a gastar muchísimas más generaciones simplemente para acercarse a valle central, perdiendo su capacidad de exploración fina (refinamiento) en las etapas finales.

DISCUSIÓN ANALÍTICA

Cálculo del Espacio de Búsqueda Binaria

Si se intentara resolver la función esfera en 10 dimensiones utilizando el Algoritmo Genético Binario con una resolución de 16 bits por variable, el cromosoma de cada individuo tendría una longitud de: $10 \text{ variables} \times 16 \text{ bits/variable} = 160 \text{ bits}$.

El tamaño total del espacio de búsqueda discreto estaría dictado por todas las combinaciones posibles de ceros y unos en esa cadena de 160 bits, lo cual equivale a: 2^{160} , alrededor de 1.4615×10^{48} combinaciones posibles.

Aunque una computadora pudiera evaluar mil millones de combinaciones por segundo, tardaría trillones de años en explorar una pequeña fracción de este espacio.

¿Por qué la codificación continua es preferible para problemas multivariados?

Con base a la cifra anterior calculada, la superioridad de la codificación continua para problemas de alta dimensionalidad se fundamenta en las siguientes razones geométricas y computacionales:

- **Eliminación de la explosión combinatoria;** En el AGC, el individuo de 10 dimensiones no es una cadena de 160 bits, sino un vector compacto de exactamente 10 números de punto flotante. El tamaño del genotipo se mantiene lineal respecto a la dimensión del problema, reduciendo drásticamente la huella de memoria y el tiempo de decodificación.
- **Navegación directa del gradiente:** El AGB es ciego a la cercanía de los números reales; cambiar un solo bit puede provocar un salto masivo y caótico de un extremo al otro del mapa de soluciones. El AGC, mediante operadores como la mutación gaussiana, realiza una exploración matemática coherente. Un individuo puede dar pasos pequeños y orgánicos alrededor de su posición actual, descendiendo suavemente hacia los valles matemáticos.
- **Resolución infinita teórica:** Para acercarse a la precisión de un número flotante de 64 bits en el AGB, se requerirían genotipos inmanejables, provocando convergencia prematura inmediata. El AGC aprovecha la representación nativa de la arquitectura del hardware, permitiendo refinamientos asintóticos sin penalizar el tamaño del cromosoma.

ANEXO DEL CÓDIGO

ga_continuo.py

"""

Autor: Mariscal Rodríguez Omar Jesús

Materia: Algoritmos Metaheurísticos

Profesor: Paredes López Ángel Ignacio

Actividad 3 - Algoritmo Genético Continuo

Universidad de Guadalajara

Centro Universitario de Ciencias Exactas e Ingenierías

ga_continuo.py

Motor de un Algoritmo Genético Continuo (representación real, punto flotante en lugar de bits como el algoritmo genético binario)

para minimización de funciones con dominio restringido tipo caja.

Arquitectura (misma idea que la Actividad 2):

- GAConfig / ProblemSpec: dataclasses que centralizan la parametrización.

- Sin "Encoding" binario: los individuos son directamente vectores de

- floats (n_vars,), acotados a su dominio. No es necesario codificar y por ende, decodificar

- Operadores (selección, cruza, mutación, manejo de límites): Protocol +

- registry, intercambiables por nombre en GAConfig sin tocar el motor.

- StoppingCriterion: paro fuerte (máx. generaciones), paro débil

(estancamiento) y un criterio opcional de diversidad poblacional

(desactivado por defecto), combinables.

- Funciones objetivo: Ackley 2D (para comparar directo con la Actividad 2)

y Esfera N-dimensional (parametrizable en número de variables).

Operadores continuos implementados (todos vectorizados con NumPy):

Cruza:

- Aritmética (whole arithmetic): combinación lineal convexa de

ambos padres con un único alpha para todo el vector.

- BLX-alpha (blend crossover): cada gen del hijo se muestrea

uniformemente en un intervalo ampliado alrededor de los padres,

lo que favorece la exploración (útil para evadir mínimos locales

en funciones multimodales como Ackley).

- Uniforme real (El visto en clase): cada gen se hereda de un padre u otro (sin mezcla),

análogo continuo del cruce uniforme discreto.

Mutación:

- Gaussiana (La vista en clase): a cada gen, con probabilidad P_m , se le suma ruido

$N(0, \sigma)$, con σ proporcional al rango de esa variable.

- Reinicio aleatorio (uniforme): con probabilidad P_m , el gen se

reemplaza por un valor aleatorio uniforme dentro de su dominio

(mutación "fuerte", útil para escapar de convergencia prematura).

Manejo de límites tras cruza/mutación:

- Clipping (default): recorte directo a la frontera del dominio.

- Reflejo: "rebota" el valor excedente hacia adentro del dominio.

- Resampling: si el valor cae fuera, se reemplaza por un valor

aleatorio uniforme dentro del dominio.

"""

```
from __future__ import annotations # Para solucionar importaciones  
circulares
```

```
from dataclasses import dataclass, field # Para contenedores de  
datos
```

```
from typing import Callable, Protocol, Sequence # Para la  
arquitectura por Patrón de Estrategia
```

```
import numpy as np # Cálculos Vecotiales Optimizados
```

```
from numpy.random import Generator # Para el manejo de números  
aleatorios y semillas
```

"""

1. Especificación del problema (función objetivo + dominio)

"""

```
@dataclass(frozen=True) # Las especificaciones del Problema son
inmutables una vez se declaran

class ProblemSpec:

    """Describe un problema de minimización con dominio tipo caja.

    Attributes:

        name: Identificador del problema (para reportes/registro).

        func: Función objetivo vectorizada: recibe (n_pop, n_vars)
y
            devuelve (n_pop,).

        bounds: Límites [(min, max), ...] por variable.

        known_optimum: Valor óptimo global analítico, si se conoce.

        n_vars: Dimensión del problema, inferida de `bounds`.

    """

    name: str

    func: Callable[[np.ndarray], np.ndarray]

    bounds: Sequence[tuple[float, float]]

    known_optimum: float | None = None

    @property
    def n_vars(self) -> int:

        return len(self.bounds)

    def _ackley(x: np.ndarray) -> np.ndarray:

        """f(x, y): función tipo Ackley, x, y"""

        xv, yv = x[:, 0], x[:, 1] #Extracción de las variables
```

```
term1 = -20.0 * np.exp(-0.2 * np.sqrt((xv**2 + yv**2) / 2.0))
#Primer término de la función

term2 = -np.exp((np.cos(2 * np.pi * xv) + np.cos(2 * np.pi *
yv)) / 2.0) #Segundo término de la función

return np.e + term1 + term2 + 20.0 #Forma final de la función
ackley
```

```
def _sphere(x: np.ndarray) -> np.ndarray:

    """Función Esfera N-dimensional:  $f(x) = \sum(x_i^2)$ . Óptimo
    global  $f(0, \dots, 0) = 0$ ."""

    return np.sum(x**2, axis=1)
```

```
def make_ackley_2d() -> ProblemSpec:

    """Construye el problema Ackley 2D, x, y en [-5, 5].

    Solo simplifica un poco la definición del código más
    adelante"""

    return ProblemSpec(name="ackley_2d", func=_ackley,
bounds=[(-5.0, 5.0), (-5.0, 5.0)], known_optimum=0.0)
```

```
def make_sphere(n_dims: int) -> ProblemSpec:

    """Construye el problema Esfera N-dimensional en x, y, ...
    n[(-5.12, 5.12), ...]"""

    bounds = [(-5.12, 5.12)] * n_dims

    return ProblemSpec(name=f"sphere_{n_dims}d", func=_sphere,
bounds=bounds, known_optimum=0.0)
```

```
"""
```

2. Configuración del algoritmo

```
"""

@dataclass

class GAConfig:

    """Hiperparámetros y opciones del GA continuo. Un único objeto
    que

        viaja por todo el motor, para facilitar cambios por el patrón
    de estrategia.

    Attributes:

        population_size: Tamaño de la población.

        max_generations: Criterio de paro "fuerte" (tope duro de
    iteraciones).

        crossover_prob: Probabilidad de cruce (Pc).

        mutation_prob: Probabilidad de mutación POR GEN (Pm).

        elitism: Número de mejores individuos que pasan sin
    alterar.

        selection: Operador de selección registrado en
    SELECTION_OPERATORS.

        tournament_k: Tamaño del torneo (si
    selection="tournament").

        crossover: Operador de cruza registrado en
    Crossover_OPERATORS.

        crossover_alpha: Parámetro alpha usado por "arithmetic" y
    "blx_alpha".

        mutation: Operador de mutación registrado en
    MUTATION_OPERATORS.

        mutation_sigma_frac: Desviación estándar de la mutación
    gaussiana,

            como FRACCIÓN del rango [low, high] de cada variable
    (para que
```

```
    escale automáticamente con el dominio de cada
problema).

    boundary_handling: Estrategia registrada en
BOUNDARY_HANDLERS para

        corregir valores fuera de dominio tras cruza/mutación.

    stagnation_generations: Paro DÉBIL: generaciones sin mejora
significativa antes de detenerse (0 = desactivado).

    stagnation_epsilon: Mejora mínima considerada
significativa.

    diversity_epsilon: Criterio de paro OPCIONAL adicional: si
la

        desviación estándar promedio de la población
(normalizada por

            el rango de cada variable) cae por debajo de este
umbral, se

            considera que la población colapsó (convergencia
prematura).

            0.0 = desactivado (default), pensado como herramienta
de

            diagnóstico/paro adicional configurable si se solicita
en vivo.

    success_tolerance: Tolerancia  $|f_{\text{encontrado}} - f_{\text{óptimo}}|$ 
para éxito cuando se conoce el límite óptimo.

    seed: Semilla para reproducibilidad.

    """

population_size: int = 60
max_generations: int = 300
crossover_prob: float = 0.9
mutation_prob: float = 0.1
elitism: int = 2
selection: str = "tournament"
```

```
tournament_k: int = 3

crossover: str = "blx_alpha"

crossover_alpha: float = 0.5

mutation: str = "gaussian"

mutation_sigma_frac: float = 0.1

boundary_handling: str = "clip"

stagnation_generations: int = 30

stagnation_epsilon: float = 1e-8

diversity_epsilon: float = 0.0

success_tolerance: float = 1e-3

seed: int | None = None

"""

3. Manejo de límites (dominio tipo caja)

-----

"""

# Protocolo/Contrato del manejo de límites

class BoundaryHandler(Protocol):

    def __call__(self, values: np.ndarray, low: np.ndarray, high:
np.ndarray, rng: Generator) -> np.ndarray:

        """Corrige `values` (n_pop, n_vars) para que respeten [low,
high] por columna."""

        ...

def handle_clip(values: np.ndarray, low: np.ndarray, high:
np.ndarray, rng: Generator) -> np.ndarray:

    """Recorte directo a la frontera del dominio (default)."""
```

```
return np.clip(values, low, high)

def handle_reflect(values: np.ndarray, low: np.ndarray, high:
np.ndarray, rng: Generator) -> np.ndarray:

    """Refleja el excedente hacia adentro del dominio (rebote),
iterando por

    si el rebote inicial aún cae fuera (poco común pero
posible)."""

    result = values.copy()

    for _ in range(3): # suficiente en la práctica para rangos
razonables

        below = result < low
        above = result > high

        if not (below.any() or above.any()):

            break

        result = np.where(below, 2 * low - result, result)
        result = np.where(above, 2 * high - result, result)

    return np.clip(result, low, high) # salvaguarda final


def handle_resample(values: np.ndarray, low: np.ndarray, high:
np.ndarray, rng: Generator) -> np.ndarray:

    """Sustituye cada valor fuera de dominio por un nuevo valor
aleatorio

    uniforme dentro de [low, high] (mantiene diversidad, evita
sesgo hacia

    la frontera que sí introduce el clipping)."""

    result = values.copy()

    out_of_bounds = (result < low) | (result > high)

    if out_of_bounds.any():
```

```
random_vals = low + rng.random(values.shape) * (high - low)

result = np.where(out_of_bounds, random_vals, result)

return result


# Listado de estrategias disponibles para el manejo de límites
BOUNDARY_HANDLERS: dict[str, BoundaryHandler] = {

    "clip": handle_clip,

    "reflect": handle_reflect,

    "resample": handle_resample,

}


"""

# 4. Operadores genéticos continuos (Protocol + registry ->
intercambiables)

_____

"""


# Contrato del Operador de Selección

class SelectionOperator(Protocol):

    def __call__(

        self, population: np.ndarray, fitness: np.ndarray,
n_select: int, rng: Generator, config: GAConfig

    ) -> np.ndarray:

        """Devuelve índices (n_select,) de individuos seleccionados
como padres."""

        ...


#Contrato del Operador de Cruce

class CrossoverOperator(Protocol):

    def __call__(
```

```
        self, parent_a: np.ndarray, parent_b: np.ndarray, rng:
Generator, config: GAConfig

    ) -> tuple[np.ndarray, np.ndarray]:

        """Cruza dos vectores reales y devuelve dos hijos (mismo
tamaño)."""

        ...

# Contrato para el Operador de Mutación

class MutationOperator(Protocol):

    def __call__(

        self, chromosome: np.ndarray, low: np.ndarray, high:
np.ndarray, rng: Generator, config: GAConfig

    ) -> np.ndarray:

        """Aplica mutación a un vector real y devuelve el vector
mutado

        (sin garantizar aún que respete [low, high]; eso lo hace el
boundary handler después)."""

        ...

# --- Selección (representación-agnóstica: opera solo sobre
fitness/índices) ---

def _fitness_from_minimization(raw_fitness: np.ndarray) ->
np.ndarray:

    """Convierte f(x) (a minimizar) en aptitud positiva para
ruleta."""

    worst = raw_fitness.max()

    return (worst - raw_fitness) + 1e-9
```

```
def select_roulette(
    population: np.ndarray, fitness: np.ndarray, n_select: int,
    rng: Generator, config: GAConfig
) -> np.ndarray:
    """Selección por ruleta (proporcional a la aptitud)."""
    adjusted = _fitness_from_minimization(fitness) # Pasar el
    fitness a una aptitud donde los más altos son los mejores (los
    mínimos)
    probs = adjusted / adjusted.sum() # Pasar a probabilidades
    return rng.choice(len(population), size=n_select, replace=True,
    p=probs) # Elección de n padres

def select_tournament(
    population: np.ndarray, fitness: np.ndarray, n_select: int,
    rng: Generator, config: GAConfig
) -> np.ndarray:
    """Selección por torneo de tamaño k (config.tournament_k)."""
    k = max(2, config.tournament_k) # Número de concursantes
    n_pop = len(population) # Número de Población
    selected = np.empty(n_select, dtype=np.int64) # Vector vacío de
    selección
    for i in range(n_select):
        contenders = rng.integers(0, n_pop, size=k) # Elección
        aleatoria de los competidores
        winner = contenders[np.argmin(fitness[contenders])] # El
        mínimo es el que gana el torneo
        selected[i] = winner # Se agrega al vector de los ganadores
    return selected

# Registro de los operadores de selección
SELECTION_OPERATORS: dict[str, SelectionOperator] = {
```

```
"roulette": select_roulette,

"tournament": select_tournament,

}

# --- Cruza continua ---

def crossover_arithmetic(
    parent_a: np.ndarray, parent_b: np.ndarray, rng: Generator,
    config: GAConfig
) -> tuple[np.ndarray, np.ndarray]:
    """Cruza aritmética:

        hijo1 = alpha * padre_a + (1 - alpha) * padre_b
        hijo2 = alpha * padre_b + (1 - alpha) * padre_a

        con un único `alpha` (config.crossover_alpha) aplicado a todo
        el vector.

        Genera siempre puntos intermedios entre los padres (buena
        explotación,

        poca exploración más allá del segmento que los une)."""
    if rng.random() >= config.crossover_prob:
        return parent_a.copy(), parent_b.copy()

    alpha = config.crossover_alpha
    child_a = alpha * parent_a + (1 - alpha) * parent_b
    child_b = alpha * parent_b + (1 - alpha) * parent_a
    return child_a, child_b

def crossover_blx_alpha(
```

```
parent_a: np.ndarray, parent_b: np.ndarray, rng: Generator,
config: GAConfig
) -> tuple[np.ndarray, np.ndarray]:

    """BLX-alpha.

    Para cada gen i, sea cmin = min(a_i, b_i), cmax = max(a_i,
b_i),

    d = cmax - cmin. Cada gen del hijo se muestrea uniformemente
en:

        [cmin - alpha*d, cmax + alpha*d]

    Es decir, no solo interpola entre los padres sino que EXTIENDE
el

    intervalo un factor alpha hacia afuera, generando genes que
pueden

    caer fuera del segmento que une a ambos padres. Esto introduce
exploración adicional, lo que ayuda a evadir mínimos locales en
funciones multimodales (ej. Ackley)."""

    if rng.random() >= config.crossover_prob:

        return parent_a.copy(), parent_b.copy()

    alpha = config.crossover_alpha

    cmin = np.minimum(parent_a, parent_b)
    cmax = np.maximum(parent_a, parent_b)

    d = cmax - cmin

    low = cmin - alpha * d
    high = cmax + alpha * d

    child_a = low + rng.random(len(parent_a)) * (high - low)
    child_b = low + rng.random(len(parent_a)) * (high - low)

    return child_a, child_b
```

```
#El por defecto por se visto en clase

def crossover_uniform_real(
    parent_a: np.ndarray, parent_b: np.ndarray, rng: Generator,
    config: GAConfig
) -> tuple[np.ndarray, np.ndarray]:
    """Cruza uniforme real: cada gen se hereda íntegro de un padre
    u otro

    (sin mezcla aritmética), con probabilidad 0.5 por gen. Análogo
    continuo

    directo del cruce uniforme discreto de la Actividad 2."""
    if rng.random() >= config.crossover_prob:
        return parent_a.copy(), parent_b.copy()

    mask = rng.integers(0, 2, size=len(parent_a)).astype(bool) #
    Probabilidad de heredar por cada gen

    child_a = np.where(mask, parent_a, parent_b) # El hijo A hereda
    del padre A de primero y cuando no se llega al porcentaje de
    mutación del padre B

    child_b = np.where(mask, parent_b, parent_a) # El hijo B hace
    exactamente lo contrario

    return child_a, child_b

# Registro con los operadores de cruce
CROSSOVER_OPERATORS: dict[str, CrossoverOperator] = {
    "arithmetic": crossover_arithmetic,
    "blx_alpha": crossover_blx_alpha,
    "uniform_real": crossover_uniform_real,
}
```

```
# --- Mutación continua ---

def mutate_gaussian(
    chromosome: np.ndarray, low: np.ndarray, high: np.ndarray, rng:
    Generator, config: GAConfig
) -> np.ndarray:
    """Mutación gaussiana: a cada gen, independientemente, con
    probabilidad
    Pm (config.mutation_prob) se le suma ruido ~ N(0, sigma_i),
    donde
    sigma_i = config.mutation_sigma_frac * (high_i - low_i)
    es decir, la magnitud de la perturbación escala con el rango de
    cada
    variable (para que el mismo Pm/sigma_frac tenga un efecto
    comparable
    en dominios de distinto tamaño, ej. Ackley [-5,5] vs Esfera
    [-5.12,5.12])."""
    mutated = chromosome.copy()

    do_mutate = rng.random(len(chromosome)) < config.mutation_prob
    # Probabilidad de mutación por cada gen

    if not do_mutate.any():
        return mutated

    sigma = config.mutation_sigma_frac * (high - low) # Calculo del
    sigma o magnitud del ruido

    noise = rng.normal(loc=0.0, scale=sigma) # Generar el ruido
    aleatoriamente con la distribución de gauss. Media en 0 y
    desviación estandar igual a sigma

    mutated[do_mutate] = mutated[do_mutate] + noise[do_mutate]

    return mutated
```

```
def mutate_uniform_reset(
    chromosome: np.ndarray, low: np.ndarray, high: np.ndarray, rng:
    Generator, config: GAConfig
) -> np.ndarray:
    """Mutación por reinicio aleatorio: con probabilidad Pm, el gen
    se
    reemplaza por completo por un valor uniforme en [low, high]. Es
    una
    mutación "fuerte" (mayor exploración que la gaussiana), útil
    para
    inyectar diversidad y escapar de una convergencia prematura."""
    mutated = chromosome.copy()
    do_mutate = rng.random(len(chromosome)) < config.mutation_prob
    if not do_mutate.any():
        return mutated

    random_vals = low + rng.random(len(chromosome)) * (high - low)
    mutated[do_mutate] = random_vals[do_mutate]

    return mutated

MUTATION_OPERATORS: dict[str, MutationOperator] = {
    "gaussian": mutate_gaussian,
    "uniform_reset": mutate_uniform_reset,
}

"""
# 5. Criterios de paro (combinables: fuerte + débil + diversidad
opcional)
```

```
"""
```

```
# Contrato para el Criterio de Paro
```

```
class StoppingCriterion(Protocol):
```

```
    def should_stop(self, generation: int, best_history:
list[float], population: np.ndarray) -> bool:
```

```
        ...
```

```
@dataclass
```

```
class MaxGenerationsStopping:
```

```
    """Criterio de paro FUERTE: tope duro de generaciones."""
```

```
    max_generations: int
```

```
    def should_stop(self, generation: int, best_history:
list[float], population: np.ndarray) -> bool:
```

```
        return generation >= self.max_generations
```

```
@dataclass
```

```
class StagnationStopping:
```

```
    """Criterio de paro DÉBIL: se detiene si el mejor fitness no
mejora
```

```
    más que `epsilon` durante `patience` generaciones
consecutivas."""
```

```
    patience: int
```

```
    epsilon: float
```

```
def should_stop(self, generation: int, best_history:
list[float], population: np.ndarray) -> bool:

    if self.patience <= 0 or len(best_history) <=
self.patience:

        return False

    recent = best_history[-(self.patience + 1):]

    improvement = recent[0] - min(recent)

    return improvement < self.epsilon


@dataclass
class DiversityStopping:

    """Criterio OPCIONAL adicional: se detiene si la desviación
estándar

    promedio de la población, normalizada por el rango de cada
variable,

    cae por debajo de `epsilon` (población colapsada / convergencia
prematura). Desactivado si epsilon <= 0."""

    epsilon: float

    bounds: np.ndarray # (n_vars, 2)

    def should_stop(self, generation: int, best_history:
list[float], population: np.ndarray) -> bool:

        if self.epsilon <= 0:

            return False

        ranges = self.bounds[:, 1] - self.bounds[:, 0]

        normalized_std = (population.std(axis=0) / ranges).mean()

        return bool(normalized_std < self.epsilon)
```

```
@dataclass

class CombinedStopping:

    """Combina varios criterios: se detiene si CUALQUIERA se
    cumple."""

    criteria: list[StoppingCriterion]

    def should_stop(self, generation: int, best_history:
list[float], population: np.ndarray) -> bool:

        return any(c.should_stop(generation, best_history,
population) for c in self.criteria)

def build_stopping_criterion(config: GAConfig, bounds: np.ndarray)
-> CombinedStopping:

    """Arma el criterio de paro combinado a partir de GAConfig."""

    return CombinedStopping(

        criteria=[

MaxGenerationsStopping(max_generations=config.max_generations),

StagnationStopping(patience=config.stagnation_generations,
epsilon=config.stagnation_epsilon),

            DiversityStopping(epsilon=config.diversity_epsilon,
bounds=bounds),

        ]

    )

"""

6. Resultado de una corrida
```

```
"""
```

```
# Contenedor de datos de una corrida
```

```
@dataclass
```

```
class RunResult:
```

```
    """Resultado de una corrida completa del GA continuo."""
```

```
    best_x: np.ndarray
```

```
    best_fitness: float
```

```
    generations_run: int
```

```
    convergence_best: list[float] = field(default_factory=list)
```

```
    convergence_mean: list[float] = field(default_factory=list)
```

```
    stopped_reason: str = ""
```

```
GenerationCallback = Callable[[int, np.ndarray, np.ndarray], None]
```

```
"""
```

```
# 7. Motor evolutivo (agnóstico de los operadores concretos)
```

```
"""
```

```
class GeneticAlgorithm:
```

```
    """Motor del Algoritmo Genético Continuo. Obtiene los  
    operadores
```

```
    concretos de los registries según `config`, por lo que cambiar  
    de
```

```
    operador (ej. en la evaluación presencial) es solo cambiar un  
    campo
```

```
    de GAConfig, sin tocar esta clase."""
```

```
def __init__(self, problem: ProblemSpec, config: GAConfig) ->
None:

    self.problem = problem # Problema

    self.config = config # Configuraciones

    self.low = np.array([b[0] for b in problem.bounds],
dtype=np.float64) # Array con los mínimos de cada variable

    self.high = np.array([b[1] for b in problem.bounds],
dtype=np.float64) # Array con los máximos de cada variable

    self.bounds_arr = np.array(problem.bounds,
dtype=np.float64) # (n_vars, 2)

    # Patrón de estrategia

    self.selection_fn = self._lookup(SELECTION_OPERATORS,
config.selection, "selección") # Selección

    self.crossover_fn = self._lookup(CROSSOVER_OPERATORS,
config.crossover, "cruza") # Cruce

    self.mutation_fn = self._lookup(MUTATION_OPERATORS,
config.mutation, "mutación") # Mutación

    self.boundary_fn = self._lookup(BOUNDARY_HANDLERS,
config.boundary_handling, "manejo de límites") # Manejo de límites

    # Verificar que la selección de estrategia existe

    @staticmethod

    def _lookup(registry: dict, key: str, label: str):

        try:

            return registry[key]

        except KeyError as e:

            raise ValueError(f"Operador de {label} '{key}' no
registrado. Disponibles: {list(registry)}") from e

    # Crear una población aleatoria
```

```
def _random_population(self, size: int, rng: Generator) ->
np.ndarray:

    return self.low + rng.random((size, self.problem.n_vars)) *
(self.high - self.low)

# Evaluar la población para obtener su fitness
def _evaluate(self, population: np.ndarray) -> np.ndarray:

    return self.problem.func(population)

# Ejecución de una corrida del algoritmo

def run(self, rng: Generator | None = None, callback:
GenerationCallback | None = None) -> RunResult:

    """Ejecuta el GA continuo completo. Ver
GeneticAlgorithm.run en

    ga_binario.py para la contraparte binaria (misma
filosofía)."""

    rng = rng if rng is not None else
np.random.default_rng(self.config.seed) # Motor aleatorio con una
semilla preconfigurada

    stopping = build_stopping_criterion(self.config,
self.bounds_arr) # Creación del criterio de paro

    population =
self._random_population(self.config.population_size, rng) #
Creación de población aleatoria

    fitness = self._evaluate(population) # Evaluar la población
aleatoria

    # Diccionarios de datos

    convergence_best: list[float] = []

    convergence_mean: list[float] = []

    generation = 0 # Número de generaciones
```

```
stopped_reason = "max_generations" # Razón por defecto de
paro

while True:

    # Obtener la media y el mejor
    convergence_best.append(float(fitness.min()))
    convergence_mean.append(float(fitness.mean()))

    # Función opcional para logs
    if callback is not None:
        callback(generation, population, fitness)

    # Bucle principal con criterios previamente definidos
    if stopping.should_stop(generation, convergence_best,
population):

        if generation >= self.config.max_generations: #
Paro por máximo de generaciones

            stopped_reason = "max_generations"

            elif self.config.diversity_epsilon > 0 and
self._diversity_collapsed(population):

                stopped_reason = "diversity_collapse" # Paro
por colapso de diversidad

            else:

                stopped_reason = "stagnation" # Paro por
estancamiento

                break

        population, fitness = self._next_generation(population,
fitness, rng) # Obtenemos la nueva generación (Método más adelante)

        generation += 1 # Conteo de generaciones
```

```
best_idx = int(np.argmax(fitness)) # El mejor individuo
registrado

# Retornamos el contenedor de datos RunResult
return RunResult(

    best_x=population[best_idx].copy(),

    best_fitness=float(fitness[best_idx]),

    generations_run=generation,

    convergence_best=convergence_best,

    convergence_mean=convergence_mean,

    stopped_reason=stopped_reason,

)

"""Diversidad Colapsada detiene el algoritmo si los individuos
son muy similares entre ellos"""

def _diversity_collapsed(self, population: np.ndarray) -> bool:

    ranges = self.bounds_arr[:, 1] - self.bounds_arr[:, 0] #
Rango en los límites de cada variable

    normalized_std = (population.std(axis=0) / ranges).mean() #
Media de la desviación estandar de la población / rangos

    return bool(normalized_std < self.config.diversity_epsilon)
# Si es menor al límite de diversidad el algoritmo para

def _next_generation(

    self, population: np.ndarray, fitness: np.ndarray, rng:
Generator

) -> tuple[np.ndarray, np.ndarray]:

    n_pop = len(population) # Número de individuos

    config = self.config # Configuraciones

    # --- Elitismo ---
```

UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS
ALGORITMOS METAHEURÍSTICOS
Prof, Ángel Ignacio Ángel Paredes

```
        elite_count = max(0, min(config.elitism, n_pop)) # El
número de elites

        elite_idx = np.argsort(fitness)[:elite_count] # Índices de
los N mejores

        new_population = [population[i].copy() for i in elite_idx]
# Pasan directo a la nueva población

        # --- Reproducción ---

        while len(new_population) < n_pop: # Hasta llenar la nueva
pobación

            parents_idx = self.selection_fn(population, fitness, 2,
rng, config) # Obtenemos el índice de los padres por el método de
selección

            parent_a, parent_b = population[parents_idx[0]],
population[parents_idx[1]] # Obtenemos el valor de los padres

            child_a, child_b = self.crossover_fn(parent_a,
parent_b, rng, config) # Hacemos la cruza

            child_a = self.mutation_fn(child_a, self.low,
self.high, rng, config) # Mutamos el hijo A

            child_b = self.mutation_fn(child_b, self.low,
self.high, rng, config) # Mutamos el hijo B

            new_population.append(child_a) # Agregamos el Hijo A

            if len(new_population) < n_pop: # Si aún cabe,
agregamos el hijo B

                new_population.append(child_b)

        new_population_arr = np.array(new_population,
dtype=np.float64) # Devolveremos otro arreglo para conservar
historial

        new_population_arr = self.boundary_fn(new_population_arr,
self.low, self.high, rng) # Aplicamos la estretegia seleccionada
para mantenernos en el límite de la caja
```

```
        new_fitness = self._evaluate(new_population_arr) #  
Evaluamos el nuevo fitness  
  
        return new_population_arr, new_fitness # Retornamos la  
población clippeada y el fitness
```

"""

8. Utilidad de éxito

"""

```
def is_success(problem: ProblemSpec, best_fitness: float,  
tolerance: float) -> bool | None:  
  
    """Determina si una corrida fue "exitosa" comparando contra el  
óptimo  
  
    analítico conocido. None si el problema no tiene óptimo  
conocido."""  
  
    if problem.known_optimum is None:  
  
        return None  
  
    return abs(best_fitness - problem.known_optimum) < tolerance
```

main.py

"""

Autor: Mariscal Rodríguez Omar Jesús

Materia: Algoritmos Metaheurísticos

Profesor: Paredes López Ángel Ignacio

Actividad 3 - Algoritmo Genético Continuo

Universidad de Guadalajara

Centro Universitario de Ciencias Exactas e Ingenierías

main.py

=====

Orquesta la Actividad 3 en dos modos:

 --mode tuning (default de exploración):

 Barrido de candidatos de (población, Pc, Pm) sobre Ackley
2D (mide

 capacidad de evadir mínimos locales / exploración) y Esfera
4D (mide

 precisión / explotación), con pocas corridas por
combinación. Genera

 una tabla comparativa para elegir manualmente la
configuración final.

 --mode final:

 Corre la configuración elegida (pasada por CLI) con >= 30
corridas

independientes sobre los 3 problemas de la actividad:
Ackley 2D,
Esfera 4D y Esfera 10D. Genera tabla estadística (mejor,
peor, media,
desv. std, tasa de éxito), curvas de convergencia promedio
por
problema, y una gráfica comparativa Esfera 4D vs 10D
(escalabilidad).

Para modificaciones al patrón de estrategia igual que la actividad
anterior::

- Cambiar operador de cruza/mutación/selección/manejo de
límites: son

flags de CLI que solo cambian GAConfig;
ga_continuo.GeneticAlgorithm

no requiere ningún otro cambio.

- Agregar un operador nuevo: se registra en el diccionario
correspondiente

dentro de ga_continuo.py (SELECTION_OPERATORS /
CROSSOVER_OPERATORS /
MUTATION_OPERATORS / BOUNDARY_HANDLERS).

Ejemplos de Uso:

```
python main.py --mode tuning

python main.py --mode final --population 60 --crossover-prob
0.9 --mutation-prob 0.1

python main.py --mode final --runs 5    # corrida rápida de
prueba

"""

from __future__ import annotations
```

```
import argparse

import csv

import dataclasses

import itertools # Importación para hacer más eficientes los bucles
para el modo de exploración o modo tuning

import os

import time

from dataclasses import dataclass


import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt

import numpy as np


from ga_continuo import (

    GAConfig,

    GeneticAlgorithm,

    ProblemSpec,

    RunResult,

    is_success,

    make_ackley_2d,

    make_sphere,

)


OUTPUT_DIR_DEFAULT = "resultados"

N_RUNS_FINAL_DEFAULT = 30
```

```
"""
```

```
Utilidades comunes
```

```
"""
```

```
# Crear los problemas con los contenedores de los problemas con las  
funciones auxiliares
```

```
def build_problems() -> dict[str, ProblemSpec]:
```

```
    return {  
        "ackley_2d": make_ackley_2d(),  
        "sphere_4d": make_sphere(4),  
        "sphere_10d": make_sphere(10),  
    }
```

```
# Correr una sola corrida
```

```
def run_single(problem: ProblemSpec, config: GAConfig, seed: int)
```

```
-> RunResult:
```

```
    ga = GeneticAlgorithm(problem, config)  
    return ga.run(rng=np.random.default_rng(seed))
```

```
# Correr varias veces el algoritmo
```

```
def run_many(problem: ProblemSpec, config: GAConfig, n_runs: int,
```

```
base_seed: int) -> list[RunResult]:
```

```
    rng_seeder = np.random.default_rng(base_seed)  
    results = []  
    for _ in range(n_runs):  
        run_seed = int(rng_seeder.integers(0, 2**31 - 1))  
        cfg = dataclasses.replace(config, seed=run_seed)  
        results.append(run_single(problem, cfg, run_seed))  
    return results
```

```
def mean_convergence_curve(runs: list[RunResult], kind: str =
"mean") -> np.ndarray:

    curves = [r.convergence_mean if kind == "mean" else
r.convergence_best for r in runs]

    max_len = max(len(c) for c in curves)

    padded = np.array([c + [c[-1]] * (max_len - len(c)) for c in
curves])

    return padded.mean(axis=0)

"""

Modo TUNING

_____

"""

@dataclass # Contenedor de datos para la configuración del modo
tuning

class TuningRow:

    population: int

    crossover_prob: float

    mutation_prob: float

    problem_name: str

    mean_best: float

    std_best: float

    success_rate: float | None

    mean_generations: float

# Para la línea de comandos, separadores por comas de reales
```

```
def parse_float_list(text: str) -> list[float]:  
    return [float(v) for v in text.split(",")]  
  
# Para la lista de comandos, separadores por comas de enteros  
def parse_int_list(text: str) -> list[int]:  
    return [int(v) for v in text.split(",")]  
  
# Ejecutar el modo tuning  
def run_tuning(args: argparse.Namespace) -> list[TuningRow]:  
    populations = parse_int_list(args.tuning_populations) # El  
    número de poblaciones a probar  
  
    pcs = parse_float_list(args.tuning_pc) # Probabilidad de cruces  
    a probar  
  
    pms = parse_float_list(args.tuning_pm) # Probabiliades de  
    mutaciones a probar  
  
    tuning_problems = {  
        "ackley_2d": make_ackley_2d(),          # exploración: evadir  
        mínimos locales  
        "sphere_4d": make_sphere(4),           # explotación:  
        precisión  
    }  
  
    rows: list[TuningRow] = [] # Lista de resultados que  
    almacenaremos  
  
    combos = list(itertools.product(populations, pcs, pms)) # Lista  
    de combinaciones que almacenamos utilizando itertolls  
  
    total = len(combos) * len(tuning_problems) # T0tal de  
    combinaciones x problemas  
  
    i = 0 # Contador
```

```
print(f"Modo TUNING: {len(combos)} combinaciones x  
{len(tuning_problems)} problemas ")  
  
    f"x {args.tuning_runs} corridas = {total *  
args.tuning_runs} ejecuciones totales\n")  
  
    for pop, pc, pm in combos: # Bucle principal que se recorre por  
    cada combinación  
  
        base_config = GAConfig( # Creación de la configuración de  
un combo  
  
            population_size=pop,  
  
            crossover_prob=pc,  
  
            mutation_prob=pm,  
  
            max_generations=args.max_generations,  
  
            selection=args.selection,  
  
            crossover=args.crossover,  
  
            mutation=args.mutation,  
  
            boundary_handling=args.boundary_handling,  
  
            elitism=args.elitism,  
  
            stagnation_generations=args.stagnation_generations,  
  
            stagnation_epsilon=args.stagnation_epsilon,  
  
            success_tolerance=args.success_tolerance,  
  
        )  
  
        for problem_name, problem in tuning_problems.items(): #  
Bucle para las impresiones de cada problema para el combo  
  
            i += 1  
  
            print(f"[{i}/{len(combos)} * len(tuning_problems)]  
pop={pop} Pc={pc} Pm={pm} "  
  
                f"| {problem_name} -> {args.tuning_runs}  
corridas...", end="", flush=True)  
  
            t0 = time.time() # Medición del tiempo
```

UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS
ALGORITMOS METAHEURÍSTICOS
Prof, Ángel Ignacio Ángel Paredes

```
runs = run_many(problem, base_config, args.tuning_runs,
base_seed=args.seed) # Correr tantas veces como pruebas pedidas

elapsed = time.time() - t0 # Tiempo de finalización

best_values = np.array([r.best_fitness for r in runs])
# Mejores valores obtenidos

successes = [is_success(problem, v,
args.success_tolerance) for v in best_values] # Cuantos tuvieron
éxito en relación al óptimo conocido

success_rate = float(np.mean(successes)) if all(s is
not None for s in successes) else None # Porcentaje de casos
exitosos

rows.append( # Agregar los resultados del
combo/problema

    TuningRow(

        population=pop,

        crossover_prob=pc,

        mutation_prob=pm,

        problem_name=problem_name,

        mean_best=float(best_values.mean()),

        std_best=float(best_values.std(ddof=1)) if
len(best_values) > 1 else 0.0,

        success_rate=success_rate,

mean_generations=float(np.mean([r.generations_run for r in runs])),

    )

)

print(f" media={rows[-1].mean_best:.5f}
({elapsed:.1f}s)")

return rows
```

```
#Impresion de resultados de tuning

def print_tuning_table(rows: list[TuningRow]) -> None:

    header = (f"{'Pob':>5} {'Pc':>5} {'Pm':>6} {'Problema':<12} "
              f"{'Media mejor':>12} {'Desv.Std':>10} {'% Éxito':>9} "
              {'Gen. media':>10})

    print("\n" + header)

    print("-" * len(header))

    for r in rows:

        success_str = f"{r.success_rate:.0%}" if r.success_rate is
not None else "N/A"

        print(

            f"{r.population:>5} {r.crossover_prob:>5.2f} "
            {r.mutation_prob:>6.2f} {r.problem_name:<12} "

            f"{r.mean_best:>12.6f} {r.std_best:>10.6f} "
            {success_str:>9} {r.mean_generations:>10.1f}"

        )

#Exportación de la tabla de prueba

def export_tuning_csv(rows: list[TuningRow], path: str) -> None:

    with open(path, "w", newline="", encoding="utf-8") as f:

        writer = csv.writer(f)

        writer.writerow(["poblacion", "Pc", "Pm", "problema",
"media_mejor", "desv_std", "tasa_exito", "gen_media"])

        for r in rows:

            writer.writerow([

                r.population, r.crossover_prob, r.mutation_prob,
r.problem_name,

                f"{r.mean_best:.10f}", f"{r.std_best:.10f}",

                f"{r.success_rate:.4f}" if r.success_rate is not
None else "N/A",
```

```
f"{r.mean_generations:.2f}",

    ])

print(f"\nTabla de sintonización exportada a: {path}")

"""

Modo FINAL

_____

"""

#Contenedor de datos para la corrida del algoritmo

@dataclass

class StatsRow:

    problem_name: str

    best: float

    worst: float

    mean: float

    std: float

    success_rate: float | None

    mean_generations: float

# Calcular estadísticas

def compute_stats(problem: ProblemSpec, runs: list[RunResult],
tolerance: float) -> StatsRow:

    values = np.array([r.best_fitness for r in runs])

    successes = [is_success(problem, v, tolerance) for v in values]

    success_rate = float(np.mean(successes)) if all(s is not None
for s in successes) else None

    return StatsRow(
```

```
problem_name=problem.name,

best=float(values.min()),

worst=float(values.max()),

mean=float(values.mean()),

std=float(values.std(ddof=1)) if len(values) > 1 else 0.0,

success_rate=success_rate,

mean_generations=float(np.mean([r.generations_run for r in
runs])),

)

# Impresiones

def print_stats_table(rows: list[StatsRow]) -> None:

    header = f"{'Problema':<12} {'Mejor':>12} {'Peor':>12}
{'Media':>12} {'Desv.Std':>12} {'% Éxito':>9} {'Gen.media':>10}"

    print("\n" + header)

    print("-" * len(header))

    for r in rows:

        success_str = f"{r.success_rate:.0%}" if r.success_rate is
not None else "N/A"

        print(

            f"{r.problem_name:<12} {r.best:>12.6f} {r.worst:>12.6f}
{r.mean:>12.6f} "

            f"{r.std:>12.6f} {success_str:>9}
{r.mean_generations:>10.1f}"

        )

# Exportar la tabla de resultados

def export_stats_csv(rows: list[StatsRow], path: str) -> None:

    with open(path, "w", newline="", encoding="utf-8") as f:

        writer = csv.writer(f)
```

UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS
ALGORITMOS METAHEURÍSTICOS
Prof, Ángel Ignacio Ángel Paredes

```
writer.writerow(["problema", "mejor", "peor", "media",
"desv_std", "tasa_exito", "gen_media"])

for r in rows:

    writer.writerow([

        r.problem_name, f"{r.best:.10f}",
f"{r.worst:.10f}", f"{r.mean:.10f}", f"{r.std:.10f}",

        f"{r.success_rate:.4f}" if r.success_rate is not
None else "N/A",

        f"{r.mean_generations:.2f}",

    ])

print(f"\nTabla estadística exportada a: {path}")

#Graficar convergencia

def plot_convergence(problem_name: str, runs: list[RunResult],
output_path: str, config_label: str) -> None:

    fig, ax = plt.subplots(figsize=(10, 6))

    mean_curve = mean_convergence_curve(runs, kind="mean")
    best_curve = mean_convergence_curve(runs, kind="best")

    ax.plot(mean_curve, label="Fitness medio poblacional (promedio
de corridas)", linewidth=1.5)

    ax.plot(best_curve, label="Mejor fitness (promedio de
corridas)", linewidth=1.5, linestyle="--")

    ax.set_xlabel("Generación")
    ax.set_ylabel("Fitness")
    ax.set_title(f"Convergencia promedio -
{problem_name}\n({config_label})")
    ax.legend(fontsize=8)
    ax.grid(True, alpha=0.3)
    fig.tight_layout()
```

```
fig.savefig(output_path, dpi=150)

plt.close(fig)

print(f"Gráfica de convergencia guardada: {output_path}")

# Graficar la convergencia promedio

def plot_sphere_scalability(
    runs_4d: list[RunResult], runs_10d: list[RunResult],
    output_path: str, config_label: str
) -> None:

    """Gráfica comparativa de convergencia promedio: Esfera 4D vs
    10D"""

    fig, ax = plt.subplots(figsize=(10, 6))

    curve_4d = mean_convergence_curve(runs_4d, kind="best")
    curve_10d = mean_convergence_curve(runs_10d, kind="best")

    ax.plot(curve_4d, label="Esfera 4D", linewidth=1.8)
    ax.plot(curve_10d, label="Esfera 10D", linewidth=1.8)
    ax.set_yscale("log")

    ax.set_xlabel("Generación")
    ax.set_ylabel("Mejor fitness (escala log, promedio de
    corridas)")

    ax.set_title(f"Escalabilidad: Esfera 4D vs
    10D\n({config_label})")

    ax.legend(fontsize=9)

    ax.grid(True, alpha=0.3, which="both")

    fig.tight_layout()

    fig.savefig(output_path, dpi=150)

    plt.close(fig)
```

```
print(f"Gráfica de escalabilidad (4D vs 10D) guardada:
{output_path}")

# Algoritmo final con los hiperparámetros elegidos
def run_final(args: argparse.Namespace) -> None:
    config = GAConfig( # Creación de las configuraciones
        population_size=args.population,
        max_generations=args.max_generations,
        crossover_prob=args.crossover_prob,
        mutation_prob=args.mutation_prob,
        elitism=args.elitism,
        selection=args.selection,
        tournament_k=args.tournament_k,
        crossover=args.crossover,
        crossover_alpha=args.crossover_alpha,
        mutation=args.mutation,
        mutation_sigma_frac=args.mutation_sigma_frac,
        boundary_handling=args.boundary_handling,
        stagnation_generations=args.stagnation_generations,
        stagnation_epsilon=args.stagnation_epsilon,
        diversity_epsilon=args.diversity_epsilon,
        success_tolerance=args.success_tolerance,
    )
    config_label = ( # Impresiones
        f"pop={config.population_size}, Pc={config.crossover_prob},
Pm={config.mutation_prob}, "
        f"sel={config.selection}, cruza={config.crossover},
mut={config.mutation}, "
        f"límites={config.boundary_handling}"
    )
```

```
print("=" * 70)

print("Actividad 3 - Algoritmo Genético Continuo (modo FINAL)")

print(f"Corridas por problema: {args.runs}")

print(config_label)

print("=" * 70)

problems = build_problems()

all_runs: dict[str, list[RunResult]] = {}

stats_rows: list[StatsRow] = []

    for name, problem in problems.items(): # Bucle principal por
cada problema

        print(f"\nEjecutando {name} ({args.runs} corridas)...",
end="", flush=True)

        t0 = time.time()

        runs = run_many(problem, config, args.runs,
base_seed=args.seed)

        elapsed = time.time() - t0

        print(f" listo ({elapsed:.1f}s)")

        all_runs[name] = runs

        stats_rows.append(compute_stats(problem, runs,
args.success_tolerance))

#Imprimir los resultados

print_stats_table(stats_rows)

#Exportar la tabla

os.makedirs(args.output_dir, exist_ok=True)

export_stats_csv(stats_rows, os.path.join(args.output_dir,
"tabla_estadistica_continuo.csv"))
```

```
for name, runs in all_runs.items():

    plot_convergence(

        name, runs, os.path.join(args.output_dir,
f"convergencia_{name}.png"), config_label

    )

plot_sphere_scalability(

    all_runs["sphere_4d"], all_runs["sphere_10d"],

    os.path.join(args.output_dir,
"escalabilidad_esfera_4d_vs_10d.png"),

    config_label,

)

print("\nExperimento final completo. Resultados en:",
os.path.abspath(args.output_dir))

"""

CLI / punto de entrada

_____

"""

def parse_args() -> argparse.Namespace:

    parser = argparse.ArgumentParser(description="Actividad 3 - GA
Continuo: tuning y experimento final")

    parser.add_argument("--mode", type=str, default="final",
choices=["tuning", "final"])

# Comunes

parser.add_argument("--seed", type=int, default=42)

parser.add_argument("--max-generations", type=int, default=300)

parser.add_argument("--elitism", type=int, default=2)
```

```
parser.add_argument("--selection", type=str,
default="tournament", choices=["roulette", "tournament"])

parser.add_argument("--tournament-k", type=int, default=3)

parser.add_argument("--crossover", type=str,
default="blx_alpha",

                        choices=["arithmetic", "blx_alpha",
"uniform_real"])

parser.add_argument("--crossover-alpha", type=float,
default=0.5)

parser.add_argument("--mutation", type=str, default="gaussian",
choices=["gaussian", "uniform_reset"])

parser.add_argument("--mutation-sigma-frac", type=float,
default=0.1)

parser.add_argument("--boundary-handling", type=str,
default="clip",

                        choices=["clip", "reflect", "resample"])

parser.add_argument("--stagnation-generations", type=int,
default=30)

parser.add_argument("--stagnation-epsilon", type=float,
default=1e-8)

parser.add_argument("--diversity-epsilon", type=float,
default=0.0)

parser.add_argument("--success-tolerance", type=float,
default=1e-3)

parser.add_argument("--output-dir", type=str,
default=OUTPUT_DIR_DEFAULT)

# Modo final

parser.add_argument("--runs", type=int,
default=N_RUNS_FINAL_DEFAULT)

parser.add_argument("--population", type=int, default=60)

parser.add_argument("--crossover-prob", type=float,
default=0.9)

parser.add_argument("--mutation-prob", type=float, default=0.1)
```

```
# Modo tuning

parser.add_argument("--tuning-runs", type=int, default=10)

parser.add_argument("--tuning-populations", type=str,
default="30,60,100")

parser.add_argument("--tuning-pc", type=str,
default="0.7,0.85,0.95")

parser.add_argument("--tuning-pm", type=str,
default="0.02,0.1,0.2")

return parser.parse_args()

# Función Principal

def main() -> None:

    args = parse_args()

    if args.mode == "tuning": # Si se hará el modo de prueba o
tuning

        rows = run_tuning(args)

        print_tuning_table(rows)

        os.makedirs(args.output_dir, exist_ok=True)

        export_tuning_csv(rows, os.path.join(args.output_dir,
"tabla_sintonizacion.csv"))

    else:

        run_final(args) # Si el modo es el final

if __name__ == "__main__":

    main()
```

CONCLUSIÓN

El desarrollo de la presente actividad confirmó empíricamente que la transición del paradigma discreto al continuo no es un simple cambio de sintaxis, más bien como un avance matemático imperativo para resolver problemas multivariantes en la ingeniería y optimización. A lo largo del experimento, se comprobó que los Algoritmos Genéticos Continuos (AGC) superan los Algoritmos Binarios al tratar directamente con el dominio real de las funciones.

Durante la fase de sintonización de parámetros (Parameter Tuning), se demostró que el éxito del algoritmo depende críticamente del equilibrio entre exploración y explotación. Se identificó que una población de 60 individuos combinada con una altísima tasa de cruce ($P_c = 0.95$) y una mutación moderada ($P_m = 0.10$) representa para este problema una buena manera de evadir los complejos mínimos locales de la función de Ackley bidimensional, al tiempo que permite un refinamiento (explotación) milimétrico para alcanzar el cero absoluto en la función Esfera de 4 dimensiones. Al someter esta configuración elegida contra la función de Ackley, el AGC destruyó los límites físicos de resolución del algoritmo binario previo, alcanzando una precisión 171 veces mayor y validando la superioridad de la mutación basada en ruido estadístico gaussiano.

Por otro lado, la prueba de escalabilidad (Esfera 4D vs. 10D) expuso la inherente dificultad topológica de los espacios de alta dimensionalidad. A pesar del incremento en el esfuerzo computacional y la predecible degradación en la tasa de éxito al pasar a 10 variables, el algoritmo continuo demostró ser estructuralmente resiliente, manteniendo una convergencia estable.

A fin de cuentas, cada algoritmo es interesante y bueno para lo que fue diseñado o pensado originalmente. Exploramos primero la alternativa binaria del algoritmo genético, indagando en sus problemas y limitaciones para los que el genético continuo ofrece ciertas ventajas que aquí exploramos.